Movie Recommendation System PRD

Student Name(s):BhaveshParmar,DivakarSharma
Roll No(s): PST-25-0027, PST-25-0038
Year & Section: 1st Year, 2-A
Project Title (as assigned):Movie Recommendation System
Tool Project Type: Data Scientist
Backend: Python, Django, Pandas, Scikit-learn,numpy, Streamlit, Sql, Pickle
Frontend: Html, css, Js, React

---

1. Problem Understanding

1.1 What is the problem statement in your own words?
Users face difficulty in choosing movies from thousands of available options. The goal of this project is to build a system that analyzes movie data and user preferences to recommend relevant movies efficiently and reduce decision fatigue.

1.2 Why does this problem exist or matter?
The problem exists because there are thousands of movies available online, and users often struggle to decide what to watch. This leads to confusion, wasted time, and poor user experience.

1.3 Key inputs and expected outputs:

🔹 ***Key Inputs***

The Movie Recommendation System takes the following inputs:

1. *Movie Dataset*
   - *Movie title*
   - *Genres*
   - *Overview*
   - *Keywords*
   - *Cast*
   - *Director*
2. *User Input*
   - *Selected movie name (for content-based filtering)*
   - *OR user preference/history*

🔹 *Expected Outputs*

The system generates:

- *A list of top N recommended movies*
- *Movies that are similar in genre, theme, or content*
- *Movie posters and ratings*

1.4 Target Users / End Users :

The primary end users of this system are movie viewers who struggle to choose movies from large online catalogs. This includes students, working professionals, and general entertainment users who want quick and relevant movie suggestions.

2. Functional Scope

2.1What are the core features you plan to build (must-haves)?

- *Movie Selection Interface (CLI/Web-based interface)*
- *Content-Based Recommendation Engine*
- *Similarity Calculation using Cosine Similarity*
- *Top-N Recommendation Display*
- *Basic Error Handling*

2.2 What stretch goals could you attempt if time permits?

- *Collaborative Filtering Implementation*
- *Movie Poster and API Integration*
- *Hybrid Recommendation System*

2.3 Which libraries or tools will you use?

Frontend-

- *HTML, CSS, JavaScript*
  Used for designing and styling the frontend interface.
- *React*
  Used to build a dynamic and interactive frontend (if building full-scale web app).

Backend-

- *Python*
  Core programming language used for building the recommendation engine.
- *Django*
  Backend web framework used to manage APIs, database interaction, and server-side logic.
- *Pandas*
  Used for data loading, cleaning, and preprocessing of the movie dataset.
- *NumPy*
  Used for numerical computations and handling similarity matrices.
- *Scikit-learn*
  Used for implementing CountVectorizer and Cosine Similarity for recommendations.
- *SQL*
  Used for storing user data, movie information, and managing database operations.
- *Streamlit*
  Used for quickly building and testing the ML web interface (prototype version).
- *Pickle*
  Used to save the trained model and similarity matrix for faster loading.

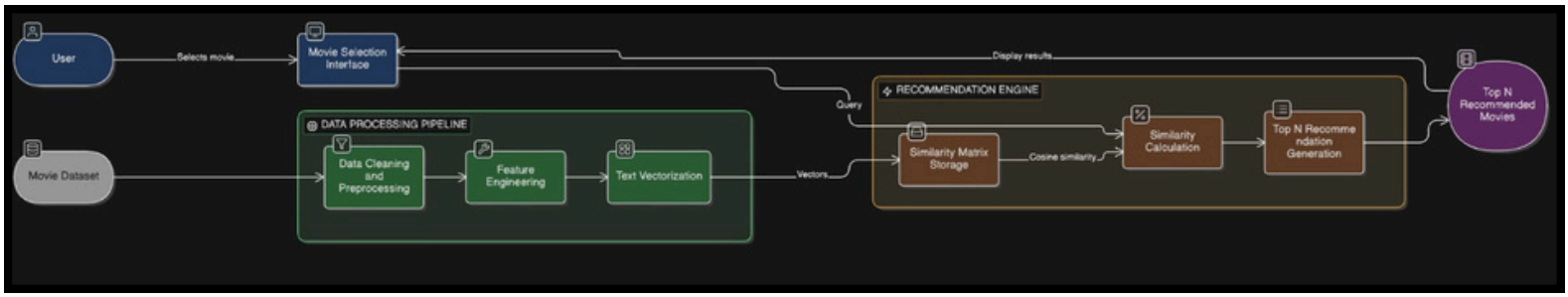
.

3. System & Design Thinking

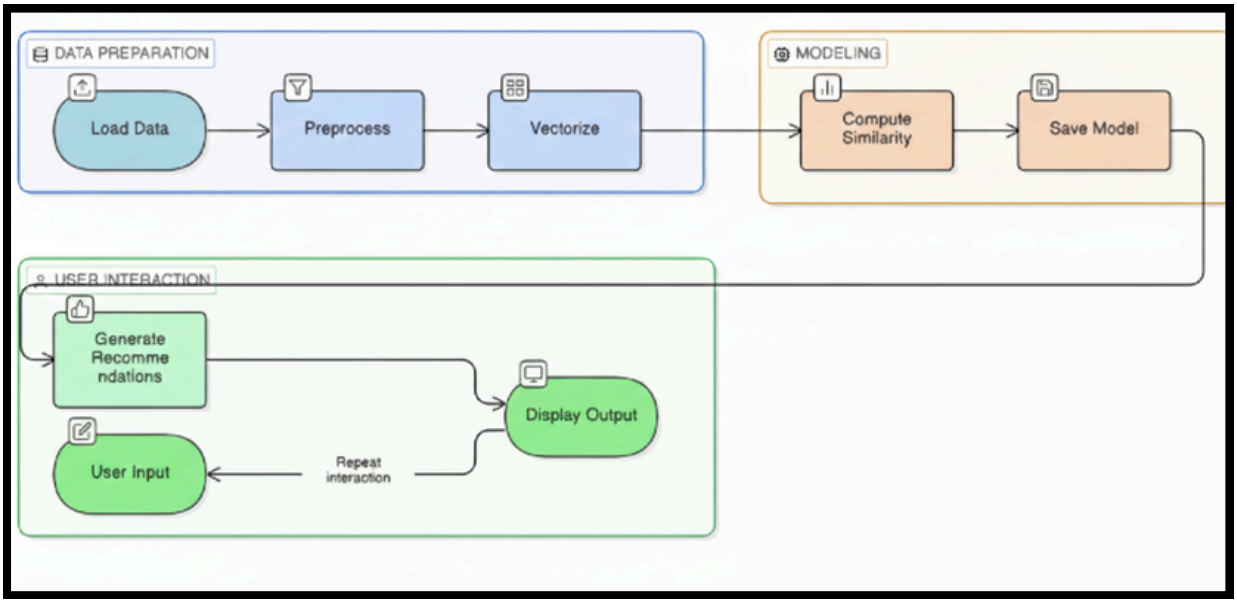
3.1Sketch or describe your appflow / pipeline:

Appflow:

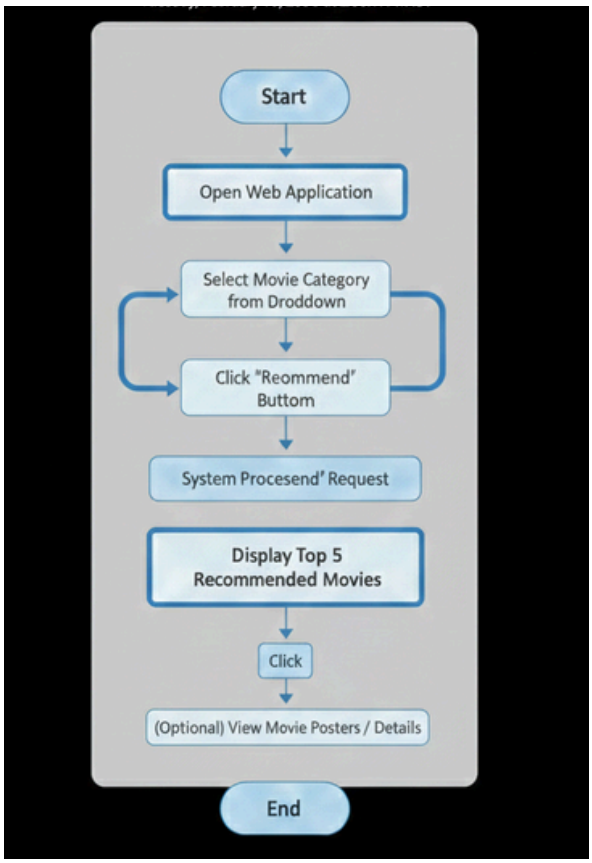
HLD :



LLD:



UserFlow :



3.2 What data structures or algorithms are central to this project?

- Data Structures
  - -DataFrame (Pandas DataFrame)
    Used to store and manipulate movie dataset (title, genres, overview, etc.).
  - -List / Array
    Used to store movie titles, similarity scores, and recommendation results.
  - -Similarity Matrix (2D Array / NumPy Array)
    Stores cosine similarity scores between all movie pairs.
- Algorithms
  - -Text Vectorization (CountVectorizer)
    Converts textual movie features into numerical vectors.
  - -Cosine Similarity Algorithm
    Measures similarity between movie vectors to find related movies.
  - -Sorting Algorithm
    Used to rank movies based on similarity scores and select top recommendations.

3.3 How will you test correctness or performance?

- 1. Functional Testing (Correctness)
  - Test with known movies and verify that recommendations are logically similar.
  - Check whether the system returns exactly Top-N results (e.g., 5 movies).
  - Validate edge cases:
    - Invalid movie name
    - Empty input
    - Missing data
- 2. Manual Validation
  - Select movies from different genres (Action, Comedy, Romance).
  - Confirm that recommended movies belong to similar genres or themes.
  - Cross-check recommendations manually for relevance.
- 3. Performance Testing
  - Measure response time for generating recommendations.
  - Ensure similarity matrix is precomputed and loaded using Pickle.
  - Test system performance with larger datasets.
- 4. Error Handling Testing
  - Provide incorrect inputs.
  - Ensure system handles errors without crashing.

4. Timeline & Milestones (12 Weeks)

Week	Planned Deliverables	Mentor Checkpoint
W1	Understand project requirements & define scope	☐
W2	Collect dataset & environment setup	☐
W3	Data cleaning & preprocessing	☐
W4	Feature selection & create tags column	☐
W5	Implement text vectorization (CountVectorizer)	☐
W6	Compute cosine similarity & test basic recommendations	☐
W7	Optimize recommendation logic & save model (Pickle)	☐
W8	Build Streamlit UI (dropdown, buttons)	☐
W9	Integrate UI with recommendation engine	☐
W10	Add enhancements (posters/API integration)	☐
W11	Testing, debugging & performance optimization	☐
W12	Documentation, final demo & project submission	☐

5. Risks & Dependencies

5.1 What's the hardest part technically for you right now?

The most technically challenging part is implementing and optimizing the similarity computation process, especially handling text preprocessing, vectorization, and ensuring accurate recommendations. Understanding how textual data is converted into numerical vectors and how cosine similarity works mathematically requires careful implementation and validation.
Additionally, integrating the backend recommendation logic with the Streamlit interface without performance issues can also be challenging.

5.2 What dependencies or help do you need from mentors?

Mentor support is required for algorithm optimization, evaluation techniques, deployment guidance, and code review.

---

6. Evaluation Readiness

6.1 How will you prove that your project "works"?

Demovideo,GitHubrepository,testlogs,screenshots

6.2 What success metric or goal will you aim for?

The goal is to generate accurate, relevant, and fast movie recommendations that improve user experience.

7. Responsibilities

7.1 Responsibilities :

Task	Bhavesh	Divakar	Mentor Notes
Requirement Analysis	✓	✓	
Dataset Collection & Cleaning	✓	✓	
Feature Engineering (Tags Creation)	✓		
Vectorization & Similarity Calculation		✓	
Recommendation Logic Implementation		✓	
UI Development (Streamlit)	✓		
Integration (UI + Backend)	✓	✓	
Testing & Debugging	✓	✓	
Documentation & Final Report		✓	